

Assignment 2

Due by 11:59 pm on 18 October 2025

Covered contents:

Lectures 4-6

Submission method:

For Questions 1 to 3, please submit your answers as a .pdf file. For Question 4, submit your answer as a file titled `histogram.py`.

A note on using generative tools:

You are welcome to use generative tools to assist you in completing assignments. However, you are not allowed to simply copy and paste; you should understand the outputs and submit works that are completed based on your own understanding. Failure to do so may result in penalties.

If you have any questions, please email phycsan@ust.hk.

Question 1 [10 points]

Investigate your own computer's CPU by answering the following questions:

- (a) Does your CPU use an x86 or ARM architecture (or something else)?
- (b) How many cores does your CPU have?
- (c) What are the sizes of your L1, L2 and L3 caches?
- (d) What is the size of a cache line on your CPU caches?
- (e) Does your CPU support simultaneous multithreading? If so, how many threads per core?
- (f) Does your CPU support SIMD?

On Linux, most of the information may be found by using the `lscpu` command in the terminal (finding the size of a cache line is a bit more complicated, and you are encouraged to look for references online). If you are on MacOS, you probably have to use `sysctl` instead.

You should provide evidence by including outputs from your terminal to each sub-question in your submitted work (i.e. don't just give a single terminal output, pinpoint which part is relevant to each sub-question).

Question 2 [20 points]

Consider the following program for computing the sum of a size- n array of floats:

```
1 # a defined here...
2 sum = 0
3
4 for i in range(n):
5     sum += a[i]
```

In this problem, assume that the code is executed in a single-core CPU without caches. The CPU has clock frequency F GHz, and infinitely many registers. The memory latency is T_ℓ nanoseconds, and the memory bandwidth is B GB/s.

Also, assume maximal latency hiding and that each addition only takes one clock cycle.

- (a) Would we benefit from having a superscalar core to execute the code above? Explain your reasoning.

- (b) Suppose the bandwidth is such that the CPU performs additions faster than the rate at which data arrives. Estimate how long it will take for the program above to terminate (consider only the for loop part).
- (c) Assume for some reasons that the data is actually arriving at a rate faster than the CPU can perform additions. How would your answer in (b) change?
- (d) How would your answer in (c) change if the CPU only has a finite number of registers? You don't need to do any calculation for this question, just provide a brief explanation.

Question 3 [10 points]

- (a) During lecture, we found the arithmetic intensity of computing the dot product between two n -dimensional vectors (consisting of floats). However, we ignored caches. Do you think including effects of caches would change our answer? Explain your reasoning.
- (b) We also considered the following naive algorithm to perform matrix multiplication between two $n \times n$ matrices (made up of floats):

```
1 # A, B, and C defined here...
2
3 for i in range(n):
4     for j in range(n):
5         for k in range(n):
6             C[i][j] += A[i][k] * B[k][j]
```

We computed the arithmetic intensity by ignoring caches. Do the same computation by including effects of caches. For simplicity, you may assume that the caches are large enough to store the entire matrices A and B , and that each entry in C is only written back to main memory once.

The next question (part (c)) is optional. It does not count towards your grades.

- (c) Can you devise an alternative matrix multiplication algorithm that can utilize caches better than our naive algorithm above?

Question 4 [20 points]

Using the `threading` module in Python, implement a parallel histogram algorithm. Your parallel histogram function should accept 5 arguments:

1. **data**: an array of numbers from which the histogram is built
2. **low**: the lower range of the bins
3. **high**: the upper range of the bins
4. **n_bins**: number of bins used
5. **n_threads**: number of threads to use

The bins (except the last) should be half-open on the right: that is, they are of the form $[a, b)$. Your histogram function should return the count of elements in each bin.

For example, `histogram(data = [0.1, 0.2, 0.4, 0.2, 0.4, 0.9, 0.2, 0.1], low=0., high=1., n_bins=10, n_threads=<whatever>)` should return `[0, 2, 3, 0, 2, 0, 0, 0, 0, 1]`.

Here's one idea on how to do it:

- Maintain a global array **results** to store final output
- Each thread works on a chunk of the data by incrementing the counts in **results**

You are free to use alternative implementations as well if you think the one above is not going to be efficient ;)

You should also include a serial implementation for measuring the speed-up on randomly generated data.

A skeleton code is included along with this assignment. You may complete the missing sections there, or you may write your own code from scratch.